

ECORIDE

Manuel d'utilisation

Application web de covoiturage écologique

Ce manuel présente l'utilisation de l'application EcoRide, ses principaux rôles et fonctionnalités, ainsi que les éléments nécessaires à son installation et à son environnement technique.

Projet EcoRide • PHP / MVC / POO • MySQL • MongoDB • Docker • GitHub • Render

1. Présentation de l'application

EcoRide est une application web de covoiturage écologique développée en PHP selon une architecture MVC/POO.

Elle permet notamment :

- La recherche et la création de trajets.
- La participation à des trajets.
- La gestion d'un système de crédits.
- La gestion des véhicules et des préférences.
- Le dépôt et la validation des avis.
- La gestion de plusieurs rôles : Utilisateur, Chauffeur, Employé et Administrateur.

L'objectif est de proposer une solution de mobilité plus responsable et collaborative.

2. Installation et lancement en local

Environnement utilisé

- PHP 8.2.12 dans l'environnement local du projet.
- XAMPP et serveur Apache.
- MySQL / MariaDB sur le port 3307.
- Composer pour les dépendances PHP.
- Extension PHP MongoDB et bibliothèque mongodb/mongodb.
- Docker pour la conteneurisation.

Lancement de l'application

- Ouvrir XAMPP.
- Démarrer Apache et MySQL.
- Vérifier que MySQL utilise le port 3307.
- Ouvrir l'application dans le navigateur.

URL locale :

`http://localhost:8080/ecoridestudi/ecoride/public/`

3. Configuration des bases de données

EcoRide utilise deux solutions de stockage complémentaires.

MySQL — base relationnelle

- Utilisée pour les données métier de l'application.
- Connexion PHP réalisée avec PDO.
- Base locale : ecoride.
- Port local : 3307.
- En production : base MySQL hébergée sur Aiven.

Les principales tables sont : utilisateurs, vehicules, preferences, trajets, participations et avis.

MongoDB Atlas — base NoSQL

- Utilisée en complément de MySQL pour la journalisation des activités de l'application.

- Connexion depuis PHP avec le driver MongoDB et la bibliothèque mongodb/mongodb.
- L'URI de connexion est stockée dans une variable d'environnement.

MySQL conserve les données métier structurées tandis que MongoDB est utilisé pour les données de journalisation.

4. Comptes de démonstration

Les identifiants ci-dessous sont conservés volontairement comme comptes de démonstration du projet.

Compte Utilisateur

Email : utilisateur@example.com

Mot de passe : utilisateur123

- Rechercher un trajet
- Réserver une place
- Annuler une réservation
- Laisser un avis
- Gérer son profil
- Consulter ses crédits

Compte Employé

Email : employe@ecoride.fr

Mot de passe : employe123

- Valider ou refuser les avis
- Consulter les signalements
- Gérer les trajets problématiques

Compte Administrateur

Email : admin@ecoride.fr

Mot de passe : admin123

- Créer des comptes employés
- Suspendre des comptes
- Visualiser les statistiques
- Gérer la plateforme

5. Recherche et création d'un trajet

Recherche de trajets

- Indiquer une ville de départ.
- Indiquer une ville d'arrivée.
- Sélectionner une date.
- Filtrer les trajets écologiques.

Les résultats sont affichés sous forme de cartes.

Création d'un trajet — Chauffeur

- Définir le départ et l'arrivée.
- Indiquer la date et l'heure.

- Définir le prix en crédits.
- Sélectionner un véhicule enregistré.
- Définir les places disponibles.
- Activer l'option écologique si souhaité.

6. Participation à un trajet

Pour participer à un trajet :

- Sélectionner le trajet souhaité.
- Consulter les détails.
- Vérifier les places disponibles et les crédits.
- Cliquer sur « Réserver ».
- La participation est enregistrée et les places disponibles sont mises à jour.

Les crédits du passager sont débités selon les règles métier prévues par l'application.

Une participation peut également être annulée selon les règles prévues par l'application.

7. Déroulement d'un trajet

Après la participation :

- Le chauffeur peut démarrer le trajet.
- L'état du trajet est mis à jour pendant son déroulement.
- Le chauffeur clôture le trajet à l'arrivée.
- Après la fin du trajet, le passager est invité à valider le trajet et à déposer un avis.
- Les crédits destinés au chauffeur sont traités après la validation prévue par le processus.

En cas de problème signalé, l'employé peut intervenir avant la mise à jour définitive des crédits.

8. Avis et signalements

Système d'avis

- Le passager peut laisser un avis après le trajet.
- L'avis est enregistré en attente de validation.
- L'employé peut valider ou refuser l'avis.
- Un avis validé peut ensuite être rendu visible.

Signalements

- Un problème peut être signalé après un trajet.
- L'employé consulte les signalements.
- Le traitement du problème peut être effectué avant la finalisation des crédits.

9. Système de crédits

- Chaque utilisateur possède un solde de crédits.
- Les crédits sont débités lors d'une participation selon les règles métier.
- Le chauffeur reçoit les crédits correspondant au trajet après la validation prévue.

- En cas de problème, le crédit peut être bloqué jusqu'au traitement.
- Une annulation peut entraîner le remboursement prévu par l'application.

10. Gestion des rôles

Les droits sont adaptés au rôle de chaque compte. Les accès sont protégés par le contrôle des sessions et des rôles.

Utilisateur : Recherche, participation, avis, profil et crédits.

Chauffeur : Proposition et gestion des trajets, participation et clôture.

Employé : Validation/refus des avis, signalements et trajets problématiques.

Administrateur : Gestion des comptes, suspensions, statistiques et plateforme.

11. JavaScript et fonctionnement asynchrone

Le projet contient du JavaScript côté interface. Un appel asynchrone avec **fetch()** est notamment utilisé dans l'espace administrateur pour transmettre une action au serveur sans rechargement complet de la page.

12. Sécurité

- Hashage des mots de passe.
- Protection des routes.
- Vérification des rôles.
- Requêtes préparées avec PDO.
- Validation des données côté serveur.
- Variables sensibles de production conservées dans l'environnement de déploiement.

13. Responsive Design

L'application est conçue pour s'adapter aux différents supports :

- Ordinateur de bureau.
- Tablette.
- Mobile.

L'adaptation est réalisée notamment avec les Media Queries CSS.

14. Déploiement et versioning

- Projet versionné avec Git.
- Dépôt GitHub.
- Application conteneurisée avec Docker.
- Application déployée sur Render.
- Base MySQL de production hébergée sur Aiven.
- Base NoSQL MongoDB Atlas utilisée pour la journalisation.

15. Structure du projet

- **app/** : architecture MVC/POO, contrôleurs, modèles et composants cœur.

- **public/** : point d'entrée de l'application et ressources publiques.
- **views/** : vues et gabarits.
- **docs/** : documentation du projet.
- **composer.json / composer.lock** : dépendances PHP.
- **Dockerfile** : configuration de la conteneurisation.

Le dossier **docs** se trouve dans le projet EcoRide.

16. Conclusion

EcoRide est une application web complète intégrant une architecture MVC/POO, une base relationnelle MySQL, une base NoSQL MongoDB pour la journalisation, un système de crédits, une gestion multi-rôles, une interface responsive, du JavaScript avec traitement asynchrone, Docker et un déploiement GitHub / Render / Aiven.